

โครงการพัฒนาระบบท่าเรืออัจฉริยะ (Digital Smart Port)

เอกสารกำหนดรายละเอียด API Specification

สำหรับการเชื่อมโยงระบบ Single Sign-On (SSO) Portal

จัดทำตามข้อกำหนด TOR 4.3.2 (Detailed Design Specification)

ชื่อเอกสาร	API Specification: SSO Portal Integration (OpenAPI + SOAP XML)
อ้างอิง TOR	TOR ข้อ 4.3.2 และภาคผนวก ก (SSO/SOAP/OpenAPI)
เวอร์ชันเอกสาร	4.0 (Merged: Main 10 Sections + Annex A-E) ปรับแก้ สารบัญ ปรับแก้ Responsibility Matrix เพิ่ม 5.1 และปรับลำดับหัวข้อ ปรับแก้ภาพ Diagram เพิ่ม ภาคผนวก E เพิ่มลำดับการทำงาน (Sequence Diagram)
วันที่จัดทำ	_____/_____/_____
ผู้จัดทำ	_____
ผู้ตรวจทาน/ผู้อนุมัติ	_____/_____

สารบัญ

1. บทนำ
 - 1.1 ขอบเขตการบังคับใช้
 - 1.2 เอกสารอ้างอิง
2. นิยามและคำย่อ
3. ภาพรวมสถาปัตยกรรมการเชื่อมโยง (Integration Overview)
 - 3.1 Logical Flow
 - 3.2 Responsibility Matrix
4. มาตรฐานทางเทคนิค (Technical Standards)
 - 4.1 Protocol และ Data Format
 - 4.2 Versioning
 - 4.3 Timeouts และ Retry Policy (แนะนำ)
5. รายละเอียด API ที่ต้องพัฒนา
 - 5.1 API ของระบบปลายทาง: TokenAcquisition (REST/JSON)
 - 5.2 API ของระบบปลายทาง: TokenAcquisition (SOAP/XML)
 - 5.3 API ของระบบปลายทาง: ValidateUser (REST/JSON)
 - 5.4 API ของระบบปลายทาง: ValidateUser (SOAP/XML)
 - 5.5 API ของ SSO Portal: IssueDeepLinkToken (REST/JSON)
 - 5.6 API ของ SSO Portal: ValidateToken (REST/JSON)
6. พฤติกรรมของระบบปลายทางเมื่อรับ Token (Token Consumer Requirements)
7. ข้อกำหนดด้านความมั่นคงปลอดภัย (Security Requirements)
8. มาตรฐานการจัดการข้อผิดพลาด (Error Handling Standard)
9. ข้อกำหนดด้านสมรรถนะและความพร้อมใช้งาน (Performance & Availability)
10. ชุดทดสอบการเชื่อมโยง (Integration Test Cases)

11. Deliverables / เอกสารส่งมอบ
12. ภาคผนวก A: OpenAPI Specification (SSO Portal API) – YAML
13. ภาคผนวก B: WSDL/XSD Skeleton – PMIS (ValidateUser SOAP Service)
14. ภาคผนวก C: WSDL/XSD Skeleton – MTP Port Net (ValidateUser SOAP Service)
15. ภาคผนวก D: WSDL/XSD Skeleton – VTMS (ValidateUser SOAP Service)

1. บทนำ

เอกสารฉบับนี้จัดทำขึ้นเพื่อกำหนดมาตรฐานและรายละเอียดของ API รับการเชื่อมโยง

ระบบงานต่าง ๆ ภายใต้โครงการ Digital Smart Port กับระบบยืนยันตัวตนแบบจุดเดียว (SSO Portal) เพื่อให้ทุกระบบสามารถตรวจสอบผู้ใช้งาน การอนุญาตการเข้าใช้งาน และการแลกเปลี่ยน

ข้อมูลการยืนยันตัวตนได้อย่างปลอดภัยและเป็นมาตรฐานเดียวกัน

1.1 ขอบเขตการบังคับใช้

- ระบบบริหารจัดการข้อมูลและการจัดเก็บรายได้ (PMIS)
- ระบบบริหารจัดการท่าเรืออุตสาหกรรมมาบตาพุด (MTP Port Net)
- ระบบควบคุมการจราจรทางน้ำ (VTMS)
- ระบบ SSO Portal (Smart Port Platform/Portal)
- ระบบอื่น ๆ ที่เกี่ยวข้องในอนาคต

1.2 เอกสารอ้างอิง

- TOR โครงการพัฒนาระบบท่าเรืออัจฉริยะ (Digital Smart Port)
- ข้อเสนอทางเทคนิค (Proposal) ที่ใช้เป็นฐานในการจัดทำ Specification
- บันทึก/การประชุมทีม DEV เพื่อหารือเรื่อง SSO (IEAT)
- แนวทาง WS-Security UsernameToken (ตัวอย่างอ้างอิงสาธารณะ เช่น StackOverflow)

2. นิยามและคำย่อ

- SSO: Single Sign-On ระบบยืนยันตัวตนแบบจุดเดียว
- IdP: Identity Provider ผู้ให้บริการยืนยันตัวตน (เช่น SAML/OIDC)
- SP: Service Provider ระบบปลายทางที่รับรอง token/assertion

- JIT Provisioning: การสร้างบัญชีผู้ใช้งานใน SSO Portal แบบ Just-in-time เมื่อมีการใช้งานครั้งแรก
- Access Token: Token สำหรับยืนยันการเข้าถึงบริการ (อายุใช้งานยาวกว่า)
- Deep Link Token: Token อายุสั้น ใช้ครั้งเดียวสำหรับการกดเข้าโปรแกรมปลายทางผ่าน Portal

3. ภาพรวมสถาปัตยกรรมการเชื่อมโยง (Integration Overview)

3.1 Logical Flow

กระบวนการเชื่อมโยงประกอบด้วย 2 ระยะหลัก:

- ระยะที่ 1: User Discovery – SSO Portal ตรวจสอบว่ามีผู้ใช้งานอยู่ในระบบใดระบบหนึ่งหรือไม่
- ระยะที่ 2: Token-based Access – SSO Portal ออก Token

เพื่อให้ระบบปลายทางตรวจสอบและอนุญาตการเข้าใช้งาน โดยไม่ส่งรหัสผ่านข้ามระบบ

3.2 Responsibility Matrix

หน่วยงาน/ระบบ	API ที่ต้องพัฒนา/ให้บริการ	บทบาท
SSO Portal	Login/Token/ValidateToken	Provider (ศูนย์กลางการออกและตรวจสอบ Token)
PMIS	TokenAcquisition ValidateUser (+ HealthCheck)	Provider (แหล่งอ้างอิงผู้ใช้งานของระบบ)
MTP Port Net	TokenAcquisition ValidateUser (+ HealthCheck)	Provider (แหล่งอ้างอิงผู้ใช้งานของระบบ)
VTMS	TokenAcquisition ValidateUser (+ HealthCheck)	Provider (แหล่งอ้างอิงผู้ใช้งานของระบบ)
ทุกระบบปลายทาง	Consume Token / Call ValidateToken	Consumer (ตรวจสอบ Token จาก SSO ก่อนอนุญาตใช้งาน)

4. มาตรฐานทางเทคนิค (Technical Standards)

4.1 Protocol และ Data Format

- REST/JSON: ใช้สำหรับ SSO Portal API (OpenAPI 3.0)
- SOAP/XML: ใช้สำหรับการเชื่อมโยงกับระบบเดิม/Legacy และตามข้อกำหนด TOR
- Character Encoding: UTF-8

4.2 Versioning

กำหนดรูปแบบเวอร์ชันของ API เป็น /api/v1/... และต้องประกาศการ

เปลี่ยนแปลงแบบไม่ทำให้ระบบเดิมใช้งานไม่ได้ (Backward-compatible)

4.3 Timeouts และ Retry Policy (แนะนำ)

- Timeout ต่อคำขอ: 2 วินาที (p95) สำหรับ ValidateUser
- Retry: 3 ครั้งแบบ Exponential backoff (1s, 2s, 4s) เฉพาะกรณี error ชั่วคราว
- Circuit breaker: เปิดเมื่อพบความล้มเหลวต่อเนื่องเกินเกณฑ์ เพื่อป้องกันกระทบทั้งระบบ

5. รายละเอียด API ที่ต้องพัฒนา

5.1 API ของระบบปลายทาง: Token Acquisition (Client Authentication) (REST/JSON)

API นี้ต้องพัฒนาโดย PMIS/MTP/VTMS เพื่อให้ SSO Portal เรียกใช้ เพิ่มความปลอดภัยในการ

เชื่อมต่อ API ทุกระบบที่เรียกใช้งานระบบปลายทาง เช่น PMIS จะต้องผ่านขั้นตอนการขอ Access

Token ก่อน โดยใช้ Client Credential (client_id และ client_secret)

- Method: POST
- Path: /api/v1/auth/token/api-dashboard

5.1.1 Request Schema

```
{  
  "client_id": "string",  
  "client_secret": "string"  
}
```

5.1.2 Response Schema (Success)

```
{  
  "access_token": "string",  
  "token_type": "string",  
  "expires_in": "number"  
}
```

5.1.3 Response Schema (Not Found)

```
{  
  "error": "string",  
  "message": "string",  
  "status_code": "string"  
}
```

5.1.4 Business Rules

- API นี้ใช้สำหรับการยืนยันตัวตนของระบบ (Client Authentication) เท่านั้น ไม่เกี่ยวข้องกับการยืนยันตัวตนของผู้ใช้งาน (User Authentication)
- ระบบที่เรียกใช้งาน API ต้องระบุ client_id และ client_secret ที่ถูกต้องทุกครั้ง
- ระบบจะออก Access Token ต่อเมื่อ client_id และ client_secret ถูกต้องเท่านั้น

- Access Token ที่ได้รับจะต้องถูกนำไปใช้ใน Authorization Header ของ API อื่น ๆ
ในรูปแบบ: Authorization: Bearer <access_token>
- Access Token มีอายุการใช้งานจำกัด (expires_in) และเมื่อหมดอายุจะต้องเรียก API นี้ใหม่เพื่อขอ Token ใหม่
- ระบบต้องไม่เรียก API อื่น เช่น ValidateUser โดยไม่มี Access Token
- ห้ามใช้ Username/Password ของผู้ใช้งานในการยืนยันตัวตนในขั้นตอนนี้ โดยการ Authentication ของผู้ใช้งานจะดำเนินการที่ SSO เท่านั้น
- ข้อมูล client_secret ต้องถูกเก็บรักษาอย่างปลอดภัย และต้องไม่เปิดเผยในที่สาธารณะหรือฝังไว้ใน frontend application - การเรียกใช้งาน API นี้ต้องกระทำผ่าน HTTPS เท่านั้น เพื่อป้องกันการรั่วไหลของข้อมูลสำคัญ
- ระบบควรมีการจัดการ error response เช่น 400 และ 401 อย่างเหมาะสม และต้องไม่ใช่ Token ที่ได้จาก request ที่ล้มเหลว
- แต่ละ client_id จะถูกใช้เป็นตัวระบุระบบ (System Identity) และต้องไม่ใช้ร่วมกันระหว่างหลายระบบ

5.2 API ของระบบปลายทาง: TokenAcquisition (SOAP/XML)

สำหรับระบบที่ต้องการเชื่อมต่อผ่าน SOAP ให้พัฒนา Web Service สำหรับการขอ Access Token โดยใช้ Client Credential (ClientId และ ClientSecret) เพื่อให้ SSO Portal ใช้ในการยืนยันตัวตนระดับระบบต่อระบบ (System-to-System Authentication)

SSO Portal ต้องเรียก API นี้ก่อนทุกครั้ง เพื่อรับ AccessToken สำหรับนำไปใช้เรียกบริการอื่นของระบบปลายทาง เช่น ValidateUser

Transport: HTTPS

Security Model: Client Credential Authentication

5.2.1 SOAP Operation

- Operation: GetToken
- SOAPAction: urn:Token:GetToken

5.2.2 SOAP Request Example

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:auth="http://ieat.go.th/sso/token">
  <soap:Body>
    <auth:GetTokenRequest>
      <auth:ClientId>partner_01</auth:ClientId>
      <auth:ClientSecret>secret_xyz</auth:ClientSecret>
      <auth:GrantType>client_credentials</auth:GrantType>
      <auth:Scope>validate_user</auth:Scope>
    </auth:GetTokenRequest>
  </soap:Body>
</soap:Envelope>
```

5.2.3 SOAP Response Example (Success)

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:auth="http://ieat.go.th/sso/token">
  <soap:Body>
    <auth:GetTokenResponse>
      <auth:AccessToken>
        eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
      </auth:AccessToken>
      <auth:TokenType>Bearer</auth:TokenType>
      <auth:ExpiresIn>300</auth:ExpiresIn>
    </auth:GetTokenResponse>
  </soap:Body>
</soap:Envelope>
```

5.2.4 SOAP Response Example (Invalid Client)

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:auth="http://ieat.go.th/sso/token">
  <soap:Body>
    <auth:GetTokenResponse>
      <auth:ErrorCode>INVALID_CLIENT</auth:ErrorCode>
      <auth:ErrorMessage>
        ClientId or ClientSecret is invalid
      </auth:ErrorMessage>
    </auth:GetTokenResponse>
  </soap:Body>
</soap:Envelope>
```

5.2.5 Business Rules

- API นี้ใช้สำหรับการยืนยันตัวตนของระบบ (Client Authentication) เท่านั้น ไม่เกี่ยวข้องกับการยืนยันตัวตนของผู้ใช้งาน (User Authentication)
- ระบบที่เรียกใช้งานต้องส่ง ClientId และ ClientSecret ที่ถูกต้องทุกครั้ง
- ระบบจะออก AccessToken ให้เฉพาะกรณีที่ ClientId และ ClientSecret ถูกต้องเท่านั้น
- AccessToken ที่ได้รับต้องถูกนำไปใช้ในการเรียก API อื่นของระบบปลายทาง เช่น ValidateUser
- AccessToken ต้องถูกส่งในรูปแบบ Bearer Token หรือส่งผ่าน ValidateUserRequest ตามที่กำหนดใน WSDL/XSD ของระบบปลายทาง
- AccessToken มีอายุจำกัดตามค่า ExpiresIn และเมื่อหมดอายุต้องเรียก GetToken ใหม่
- ห้ามใช้ Username/Password ของผู้ใช้งานในขั้นตอนนี้ เพราะการ Authentication ของผู้ใช้งานต้องดำเนินการที่ SSO IdP เท่านั้น

- ClientSecret ต้องถูกจัดเก็บอย่างปลอดภัย และต้องไม่เปิดเผยใน frontend application หรือระบบสาธารณะ
- ทุกการเรียกใช้งานต้องใช้ HTTPS (TLS 1.2 หรือสูงกว่า)
- ระบบต้องมีการจัดการ Error Response และต้องไม่ใช่ Token ที่ได้จาก Request ที่ล้มเหลว
- แต่ละ ClientId ใช้เป็นตัวระบุระบบ (System Identity) และต้องไม่ใช้ร่วมกันระหว่างหลายระบบ
- ระบบควรบันทึก Audit Log ของการขอ Token เช่น ClientId, Timestamp, Source IP และ Result ของการร้องขอ
- ระบบต้องไม่บันทึก ClientSecret หรือ AccessToken ลงใน Log

5.3 API ของระบบปลายทาง: ValidateUser (REST/JSON)

API นี้ต้องพัฒนาโดย PMIS/MTP/VTMS เพื่อให้ SSO Portal เรียกตรวจสอบผู้ใช้งาน (User Discovery)

Method: POST

Path: /api/v1/user/validate

Content-Type: application/json

Authorization: Bearer <access_token>

5.3.1 Request Schema

```
{
  "username": "string"
}
```

5.3.2 Response Schema (Success)

```
{
  "exists": true,
  "user_id": "string",
  "roles": ["string"],
```

```
"system": "PMIS|MTP|VTMS",  
"attributes": {"org": "string", "department": "string"}  
}
```

5.3.3 Response Schema (Not Found)

```
{  
  "exists": false,  
  "code": "USER_NOT_FOUND",  
  "message": "User not found"  
}
```

5.3.4 Business Rules

- ต้องไม่คืนข้อมูลรหัสผ่านหรือข้อมูลลับ
- ต้องคืน roles ตามที่ระบบปลายทางกำหนด (เพื่อให้ SSO Portal ทำ Role Mapping ต่อไป)
- ต้องบันทึก Integration Log (request/response metadata)

5.4 API ของระบบปลายทาง: ValidateUser (SOAP/XML)

สำหรับระบบที่ต้องการเชื่อมต่อผ่าน SOAP ให้พัฒนา Web Service ตาม WSDL โดย SSO Portal ต้องเรียก Token Acquisition ตามข้อ

5.4.1 SOAP Operation

Operation: ValidateUser

SOAPAction: urn:PMIS:ValidateUser

Security: AccessToken from Token Acquisition API

Transport: HTTPS

5.4.2 SOAP Request Example

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"  
xmlns:pmis="http://ieat.go.th/pmis/identity">
```

```
<soap:Body>
```

```
<pmis:ValidateUserRequest>
```

```
<pmis:AccessToken> eyJhbGciOiJIUzI1NiIs...
```

```
</pmis:AccessToken>
```

```
<pmis:Username>user01</pmis:Username>
```

```
</pmis:ValidateUserRequest>
```

```
</soap:Body>
```

```
</soap:Envelope>
```

5.4.3 SOAP Response Example

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:pmis="http://ieat.go.th/pmis/identity">
  <soap:Body>
    <pmis:ValidateUserResponse>
      <pmis:Exists>true</pmis:Exists>
      <pmis:UserId>12345</pmis:UserId>
      <pmis:Roles>IEAT_STAFF,PMIS_USER</pmis:Roles>
    </pmis:ValidateUserResponse>
  </soap:Body>
</soap:Envelope>
```

5.5 API ของ SSO Portal: IssueDeepLinkToken (REST/JSON)

- Method: POST
- Path: /api/v1/auth/token

5.5.1 Request Schema

```
{
  "system_id": "PMIS|MTP|VTMS",
  "target_url": "string (optional)"
}
```

5.5.2 Response Schema

```
{
  "token": "JWT_TOKEN",
  "expires_in": 300,
  "one_time_use": true,
  "redirect_url": "https://target/?token=..."
}
```

```
}
```

5.5.3 Business Rules:

Token ต้องมีอายุไม่เกิน 5 นาที และต้องใช้ได้เพียงครั้งเดียว

5.6 API ของ SSO Portal: ValidateToken (REST/JSON)

- Method: POST
- Path: /api/v1/auth/validate

5.6.1 Request Schema

```
{  
  
  "token": "JWT_TOKEN",  
  "system_id": "PMIS|MTP|VTMS"  
}
```

5.6.2 Response Schema (Valid)

```
{  
  
  "valid": true,  
  "username": "user01",  
  "roles": ["IEAT_STAFF", "PMIS_USER"],  
  "expires_at": "2026-05-15T12:00:00+07:00",  
  "one_time_use": true  
}
```

5.6.3 Response Schema (Invalid)

```
{  
  
  "valid": false,  
  "reason": "TOKEN_EXPIRED|INVALID_TOKEN|TOKEN_USED"  
}
```

6. พฤติกรรมของระบบปลายทางเมื่อรับ Token (Token Consumer Requirements)

- รับ Token และเรียก SSO ValidateToken
- หาก Token valid ให้สร้าง session ภายในระบบปลายทาง และ map role/permission
- หากเป็น one-time token ให้แจ้ง SSO เพื่อ mark token used (ถ้ามีฟังก์ชันนี้)
- หาก Token invalid ให้ปฏิเสธการเข้าใช้งานและแสดงข้อความที่เหมาะสม

7. ข้อกำหนดด้านความมั่นคงปลอดภัย (Security Requirements)

7.1 Transport Security

- ทุกการสื่อสารต้องใช้ HTTPS (TLS 1.2 หรือสูงกว่า)

7.2 SOAP Security

- ใช้ WS-Security UsernameToken (แนะนำ PasswordDigest + Nonce + Created) และควรใช้งานร่วมกับ HTTPS

7.3 Token Policy

- Access Token: อายุใช้งานตามนโยบายของ SSO (เช่น 8 ชั่วโมง)
- Deep Link Token: อายุไม่เกิน 5 นาที และ one-time use
- ต้องมีการตรวจสอบ expiration และ replay protection สำหรับ one-time token

7.4 Audit Logging

- SSO Portal ต้องบันทึก Login history, Token issuance, Token validation, Integration call history
- ระบบปลายทางต้องบันทึก Access log และผลการตรวจสอบ token

8. มาตรฐานการจัดการข้อผิดพลาด (Error Handling Standard)

กำหนดรหัสข้อผิดพลาดที่เป็นมาตรฐานกลางเพื่อให้ติดตามและแก้ไขได้ง่าย

Code	Description	Applicable API
USER_NOT_FOUND	ไม่พบผู้ใช้งานในระบบปลายทาง	ValidateUser
INVALID_REQUEST	รูปแบบคำขอไม่ถูกต้อง/ข้อมูลไม่ครบ	All
TOKEN_EXPIRED	Token หมดอายุ	ValidateToken
TOKEN_USED	Token แบบ one-time ถูกใช้แล้ว	ValidateToken
INVALID_TOKEN	Token ไม่ถูกต้อง/ลายเซ็นไม่ผ่าน	ValidateToken
SYSTEM_ERROR	ข้อผิดพลาดภายในระบบ	All

9. ข้อกำหนดด้านสมรรถนะและความพร้อมใช้งาน (Performance & Availability)

- ValidateUser response time: $p95 \leq 2$ วินาที
- รองรับการเรียกพร้อมกันจาก SSO (parallel call)
- Availability เฉลี่ย $\geq 99.5\%$ สำหรับบริการสำคัญ

10. ชุดทดสอบการเชื่อมโยง (Integration Test Cases)

ตัวอย่างกรณีทดสอบขั้นต่ำที่ต้องดำเนินการใน SIT/UAT

- TC-01: ผู้ใช้งานมีอยู่ใน PMIS → SSO สร้างผู้ใช้งานแบบ JIT และอนุญาตเข้า PMIS ผ่าน deep link token
- TC-02: ผู้ใช้งานไม่มีอยู่ในทุกระบบ → SSO ปฏิเสธ/แจ้งให้ติดต่อผู้ดูแล
- TC-03: deep link token หมดอายุ → ระบบปลายทางปฏิเสธและให้กลับไป login ใหม่
- TC-04: one-time token ถูกใช้ซ้ำ → ปฏิเสธด้วย TOKEN_USED
- TC-05: ระบบปลายทาง 1 ระบบไม่พร้อมใช้งาน → SSO ใช้ผลจากระบบอื่นตาม policy และแสดงสถานะระบบ

- TC-06: SOAP WS-Security header ไม่ถูกต้อง → ปฏิเสธการเรียกบริการและบันทึก log

11. Deliverables / เอกสารส่งมอบ

- OpenAPI Specification (YAML/JSON) สำหรับ SSO Portal
- WSDL/XSD สำหรับ SOAP ValidateUser (ต่อระบบ)
- คู่มือการติดตั้ง/ตั้งค่า Integration Registry (Endpoint, Credential, WSDL URL)
- รายงานผลการทดสอบ SIT/UAT สำหรับ SSO Integration

12. ลำดับการทำงาน

1. User Login ที่ SSO Portal
2. SSO Redirect ไป SSO IdP
3. SSO IdP Authenticate User
4. IdP ส่ง Identity Assertion กลับ SSO
5. SSO เรียก GetToken SOAP ของ PMIS/MTP/VTMS
 - ใช้ ClientId / ClientSecret
 - เพื่อขอ API Access Token ของระบบนั้น
6. PMIS/MTP/VTMS ออก API Access Token กลับมา
7. SSO ใช้ API Access Token ไปเรียก ValidateUser
 - ตรวจสอบว่าผู้ใช้นี้มีอยู่ในระบบปลายทางหรือไม่
8. PMIS/MTP/VTMS ตอบข้อมูล User / Roles
9. SSO ทำ Identity Mapping
10. SSO แสดงระบบที่ User เข้าได้
11. User เลือกระบบปลายทาง

12. SSO ออก User SSO Token

- ใช้แทน password
- ใช้สำหรับ SSO Login เข้า Target System

13. Redirect ไป Target System พร้อม User SSO Token

14. Target System เรียก ValidateToken ที่ SSO

15. SSO ยืนยัน Token

16. Target System ตรวจสอบ Authorisation

17. Access Granted / Denied

หมายเหตุ

GetToken = Client Authentication Token

ใช้สำหรับ System-to-System API Security

SSO Issue Token = User Authentication Token

ใช้สำหรับยืนยันตัวผู้ใช้งานแทน Password ตอนเข้า Target System

ภาคผนวก A: OpenAPI Specification (SSO Portal API) – YAML

ภาคผนวกนี้จัดทำเพื่อแนบสเปก OpenAPI ให้ทีมพัฒนาระบบ SSO Portal และระบบที่เกี่ยวข้องนำไปใช้งาน/นำเข้าเครื่องมือ API Management ได้โดยตรง

openapi: 3.0.4

info:

title: Smart Port SSO Portal API

version: 1.0.0

description: API สำหรับ SSO Portal: login / token issuance (deep link) / token validation

servers:

- url: https://sso.ieatsmart.go.th

tags:

- name: Auth

description: Authentication & Token

paths:

/api/v1/auth/login:

post:

tags: [Auth]

summary: Login (รับผลจาก IdP แล้วสร้าง Session/Access Token)

requestBody:

required: true

content:

application/json:

schema:

\$ref: '#/components/schemas/LoginRequest'

responses:

'200':

description: Login success

content:

application/json:

schema:

\$ref: '#/components/schemas/LoginResponse'

'400': {description: Bad request}

'401': {description: Authentication failed}

/api/v1/auth/token:

post:

tags: [Auth]

summary: Issue Deep Link Token (one-time use)

security:

- bearerAuth: []

requestBody:

required: true

content:

application/json:

schema:

\$ref: '#/components/schemas/IssueDeepLinkTokenRequest'

responses:

'200':

description: Deep link token issued

content:

application/json:

schema:

\$ref: '#/components/schemas/IssueDeepLinkTokenResponse'

'400': {description: Bad request}

'401': {description: Unauthorized}

'403': {description: Forbidden}

/api/v1/auth/validate:

post:

tags: [Auth]

summary: Validate Token (สำหรับระบบปลายทางเรียกตรวจสอบ)

requestBody:

required: true

content:

application/json:

schema:

\$ref: '#/components/schemas/ValidateTokenRequest'

responses:

'200':

description: Token valid/invalid result

content:

application/json:

schema:

\$ref: '#/components/schemas/ValidateTokenResponse'

'400': {description: Bad request}

components:

securitySchemes:

bearerAuth:

type: http

scheme: bearer

bearerFormat: JWT

schemas:

LoginRequest:

type: object

required: [grant_type]

properties:

grant_type:

```
    type: string
    description: saml2_bearer | oidc_token
saml_response:
    type: string
id_token:
    type: string
LoginResponse:
    type: object
    required: [access_token, token_type, expires_in, session_id]
    properties:
        access_token: {type: string}
        token_type: {type: string, example: Bearer}
        expires_in: {type: integer}
        session_id: {type: string}
IssueDeepLinkTokenRequest:
    type: object
    required: [system_id]
    properties:
        system_id: {type: string}
        target_url: {type: string}
IssueDeepLinkTokenResponse:
    type: object
    required: [deeplink_token, token_type, expires_in, one_time_use]
    properties:
        deeplink_token: {type: string}
        token_type: {type: string, example: Bearer}
        expires_in: {type: integer}
        one_time_use: {type: boolean}
        redirect_url: {type: string}
ValidateTokenRequest:
```


type: object
required: [token, system_id]
properties:
 token: {type: string}
 system_id: {type: string}

ValidateTokenResponse:

type: object
required: [active]
properties:
 active: {type: boolean}
 username: {type: string}
 roles:
 type: array
 items: {type: string}
 exp: {type: string, format: date-time}
 one_time_use: {type: boolean}
 reason: {type: string}

ภาคผนวก B: WSDL/XSD Skeleton – PMIS (ValidateUser SOAP Service)

โครงสร้าง WSDL/XSD สำหรับบริการ ValidateUser ของระบบ PMIS เพื่อให้ SSO Portal เรียกตรวจสอบผู้ใช้งานผ่าน SOAP/XML โดย SSO Portal ต้องเรียก Token Acquisition Service ของ PMIS ก่อน เพื่อรับ AccessToken และนำ AccessToken ดังกล่าวส่งมาที่ ValidateUserRequest ทุกครั้ง

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<definitions name="PMISIdentityService"
```

```
  targetNamespace="http://ieat.go.th/pmis/identity"
```

```
  xmlns:tns="http://ieat.go.th/pmis/identity"
```

```
  xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
```

```
  xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/"
```

```
  xmlns:xsd="http://www.w3.org/2001/XMLSchema">
```

```
<wsdl:types>
```

```
  <xsd:schema targetNamespace="http://ieat.go.th/pmis/identity"
    elementFormDefault="qualified">
```

```
    <xsd:element name="ValidateUserRequest">
```

```
<xsd:complexType>

  <xsd:sequence>

    <xsd:element name="AccessToken" type="xsd:string"/>

    <xsd:element name="Username" type="xsd:string"/>

  </xsd:sequence>

</xsd:complexType>

</xsd:element>

<xsd:element name="ValidateUserResponse">

  <xsd:complexType>

    <xsd:sequence>

      <xsd:element name="Exists" type="xsd:boolean"/>

      <xsd:element name="UserId" type="xsd:string" minOccurs="0"/>

      <xsd:element name="Roles" type="xsd:string" minOccurs="0"/>

      <xsd:element name="ErrorCode" type="xsd:string" minOccurs="0"/>

      <xsd:element name="ErrorMessage" type="xsd:string" minOccurs="0"/>

    </xsd:sequence>

  </xsd:complexType>

</xsd:element>
```

<xsd:element name="HealthCheckRequest" type="xsd:string"/>

<xsd:element name="HealthCheckResponse" type="xsd:string"/>

</xsd:schema>

</wsdl:types>

<wsdl:message name="ValidateUserInput">

<wsdl:part name="parameters" element="tns:ValidateUserRequest"/>

</wsdl:message>

<wsdl:message name="ValidateUserOutput">

<wsdl:part name="parameters" element="tns:ValidateUserResponse"/>

</wsdl:message>

<wsdl:message name="HealthCheckInput">

<wsdl:part name="parameters" element="tns:HealthCheckRequest"/>

</wsdl:message>

<wsdl:message name="HealthCheckOutput">

<wsdl:part name="parameters" element="tns:HealthCheckResponse"/>

</wsdl:message>

<wsdl:portType name="PMISIdentityPortType">

<wsdl:operation name="ValidateUser">

<wsdl:input message="tns:ValidateUserInput"/>

<wsdl:output message="tns:ValidateUserOutput"/>

</wsdl:operation>

<wsdl:operation name="HealthCheck">

<wsdl:input message="tns:HealthCheckInput"/>

<wsdl:output message="tns:HealthCheckOutput"/>

</wsdl:operation>

</wsdl:portType>

<wsdl:binding name="PMISIdentitySoapBinding" type="tns:PMISIdentityPortType">

<soap:binding style="document"

transport="http://schemas.xmlsoap.org/soap/http"/>

```
<wsdl:operation name="ValidateUser">  
  
  <soap:operation soapAction="urn:PMIS:ValidateUser"/>  
  
  <wsdl:input>  
  
    <soap:body use="literal"/>  
  
  </wsdl:input>  
  
  <wsdl:output>  
  
    <soap:body use="literal"/>  
  
  </wsdl:output>  
  
</wsdl:operation>
```

```
<wsdl:operation name="HealthCheck">  
  
  <soap:operation soapAction="urn:PMIS:HealthCheck"/>  
  
  <wsdl:input>  
  
    <soap:body use="literal"/>  
  
  </wsdl:input>  
  
  <wsdl:output>  
  
    <soap:body use="literal"/>  
  
  </wsdl:output>
```

</wsdl:operation>

</wsdl:binding>

<wsdl:service name="PMISIdentityService">

<wsdl:port name="PMISIdentitySoapPort"
binding="tns:PMISIdentitySoapBinding">

<soap:address
location="https://pmis.example.go.th/soap/PMISIdentityService"/>

</wsdl:port>

</wsdl:service>

</definitions>

หมายเหตุ: ก่อนเรียกใช้ ValidateUser SOAP Service ระบบ SSO Portal ต้องเรียก Token Acquisition Service ของ PMIS เพื่อรับ AccessToken ก่อนทุกครั้ง และต้องส่ง AccessToken มากับ ValidateUserRequest โดยใช้ร่วมกับ HTTPS เท่านั้น ห้ามใช้ Username/Password ของผู้ใช้งานหรือ WS-Security UsernameToken ในขั้นตอนนี้

ภาคผนวก C: WSDL/XSD Skeleton – MTP Port Net (ValidateUser SOAP Service)

โครงสร้าง WSDL/XSD สำหรับบริการ ValidateUser ของระบบ MTP เพื่อให้ SSO Portal เรียกตรวจสอบผู้ใช้งานผ่าน SOAP/XML โดย SSO Portal ต้องเรียก Token Acquisition Service ของ MTP ก่อน เพื่อรับ AccessToken และนำ AccessToken ดังกล่าวส่งมาที่ ValidateUserRequest ทุกครั้ง

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<definitions name="MTPIdentityService"
```

```
  targetNamespace="http://ieat.go.th/mtp/identity"
```

```
  xmlns:tns="http://ieat.go.th/mtp/identity"
```

```
  xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
```

```
  xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/"
```

```
  xmlns:xsd="http://www.w3.org/2001/XMLSchema">
```

```
<wsdl:types>
```

```
  <xsd:schema targetNamespace="http://ieat.go.th/mtp/identity"
    elementFormDefault="qualified">
```

```
    <xsd:element name="ValidateUserRequest">
```



```
<xsd:complexType>

  <xsd:sequence>

    <xsd:element name="AccessToken" type="xsd:string"/>

    <xsd:element name="Username" type="xsd:string"/>

  </xsd:sequence>

</xsd:complexType>

</xsd:element>


<xsd:element name="ValidateUserResponse">

  <xsd:complexType>

    <xsd:sequence>

      <xsd:element name="Exists" type="xsd:boolean"/>

      <xsd:element name="UserId" type="xsd:string" minOccurs="0"/>

      <xsd:element name="Roles" type="xsd:string" minOccurs="0"/>

      <xsd:element name="ErrorCode" type="xsd:string" minOccurs="0"/>

      <xsd:element name="ErrorMessage" type="xsd:string" minOccurs="0"/>

    </xsd:sequence>

  </xsd:complexType>

</xsd:element>
```

<xsd:element name="HealthCheckRequest" type="xsd:string"/>

<xsd:element name="HealthCheckResponse" type="xsd:string"/>

</xsd:schema>

</wsdl:types>

<wsdl:message name="ValidateUserInput">

<wsdl:part name="parameters" element="tns:ValidateUserRequest"/>

</wsdl:message>

<wsdl:message name="ValidateUserOutput">

<wsdl:part name="parameters" element="tns:ValidateUserResponse"/>

</wsdl:message>

<wsdl:message name="HealthCheckInput">

<wsdl:part name="parameters" element="tns:HealthCheckRequest"/>

</wsdl:message>

<wsdl:message name="HealthCheckOutput">

<wsdl:part name="parameters" element="tns:HealthCheckResponse"/>

</wsdl:message>

<wsdl:portType name="MTPIIdentityPortType">

<wsdl:operation name="ValidateUser">

<wsdl:input message="tns:ValidateUserInput"/>

<wsdl:output message="tns:ValidateUserOutput"/>

</wsdl:operation>

<wsdl:operation name="HealthCheck">

<wsdl:input message="tns:HealthCheckInput"/>

<wsdl:output message="tns:HealthCheckOutput"/>

</wsdl:operation>

</wsdl:portType>

<wsdl:binding name="MTPIIdentitySoapBinding" type="tns:MTPIIdentityPortType">

<soap:binding style="document"

transport="http://schemas.xmlsoap.org/soap/http"/>

```
<wsdl:operation name="ValidateUser">  
  
  <soap:operation soapAction="urn:MTP:ValidateUser"/>  
  
  <wsdl:input>  
  
    <soap:body use="literal"/>  
  
  </wsdl:input>  
  
  <wsdl:output>  
  
    <soap:body use="literal"/>  
  
  </wsdl:output>  
  
</wsdl:operation>
```

```
<wsdl:operation name="HealthCheck">  
  
  <soap:operation soapAction="urn:MTP:HealthCheck"/>  
  
  <wsdl:input>  
  
    <soap:body use="literal"/>  
  
  </wsdl:input>  
  
  <wsdl:output>  
  
    <soap:body use="literal"/>  
  
  </wsdl:output>
```

</wsdl:operation>

</wsdl:binding>

<wsdl:service name="MTPIIdentityService">

<wsdl:port name="MTPIIdentitySoapPort" binding="tns:MTPIIdentitySoapBinding">

<soap:address

location="https://mtp.example.go.th/soap/MTPIIdentityService"/>

</wsdl:port>

</wsdl:service>

</definitions>

หมายเหตุ: ก่อนเรียกใช้ ValidateUser SOAP Service ระบบ SSO Portal ต้องเรียก Token Acquisition Service ของ MTP เพื่อรับ AccessToken ก่อนทุกครั้ง และต้องส่ง AccessToken มากับ ValidateUserRequest โดยใช้ร่วมกับ HTTPS เท่านั้น ห้ามใช้ Username/Password ของผู้ใช้งานหรือ WS-Security UsernameToken ในขั้นตอนนี้

ภาคผนวก D: WSDL/XSD Skeleton – VTMS (ValidateUser SOAP Service)

โครงสร้าง WSDL/XSD สำหรับบริการ ValidateUser ของระบบ VTMS เพื่อให้ SSO Portal เรียกตรวจสอบผู้ใช้งานผ่าน SOAP/XML โดย SSO Portal ต้องเรียก Token Acquisition Service ของ VTMS ก่อน เพื่อรับ AccessToken และนำ AccessToken ดังกล่าวส่งมากับ ValidateUserRequest ทุกครั้ง

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<definitions name="VTMSIdentityService"
```

```
  targetNamespace="http://ieat.go.th/vtms/identity"
```

```
  xmlns:tns="http://ieat.go.th/vtms/identity"
```

```
  xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
```

```
  xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/"
```

```
  xmlns:xsd="http://www.w3.org/2001/XMLSchema">
```

```
<wsdl:types>
```

```
  <xsd:schema targetNamespace="http://ieat.go.th/vtms/identity"
    elementFormDefault="qualified">
```

```
    <xsd:element name="ValidateUserRequest">
```

```
<xsd:complexType>

  <xsd:sequence>

    <xsd:element name="AccessToken" type="xsd:string"/>

    <xsd:element name="Username" type="xsd:string"/>

  </xsd:sequence>

</xsd:complexType>

</xsd:element>

<xsd:element name="ValidateUserResponse">

  <xsd:complexType>

    <xsd:sequence>

      <xsd:element name="Exists" type="xsd:boolean"/>

      <xsd:element name="UserId" type="xsd:string" minOccurs="0"/>

      <xsd:element name="Roles" type="xsd:string" minOccurs="0"/>

      <xsd:element name="ErrorCode" type="xsd:string" minOccurs="0"/>

      <xsd:element name="ErrorMessage" type="xsd:string" minOccurs="0"/>

    </xsd:sequence>

  </xsd:complexType>

</xsd:element>
```

<xsd:element name="HealthCheckRequest" type="xsd:string"/>

<xsd:element name="HealthCheckResponse" type="xsd:string"/>

</xsd:schema>

</wsdl:types>

<wsdl:message name="ValidateUserInput">

<wsdl:part name="parameters" element="tns:ValidateUserRequest"/>

</wsdl:message>

<wsdl:message name="ValidateUserOutput">

<wsdl:part name="parameters" element="tns:ValidateUserResponse"/>

</wsdl:message>

<wsdl:message name="HealthCheckInput">

<wsdl:part name="parameters" element="tns:HealthCheckRequest"/>

</wsdl:message>

<wsdl:message name="HealthCheckOutput">

<wsdl:part name="parameters" element="tns:HealthCheckResponse"/>

</wsdl:message>

<wsdl:portType name="VTMSIdentityPortType">

<wsdl:operation name="ValidateUser">

<wsdl:input message="tns:ValidateUserInput"/>

<wsdl:output message="tns:ValidateUserOutput"/>

</wsdl:operation>

<wsdl:operation name="HealthCheck">

<wsdl:input message="tns:HealthCheckInput"/>

<wsdl:output message="tns:HealthCheckOutput"/>

</wsdl:operation>

</wsdl:portType>

<wsdl:binding name="VTMSIdentitySoapBinding"
type="tns:VTMSIdentityPortType">

```
<soap:binding style="document"
transport="http://schemas.xmlsoap.org/soap/http"/>
```

```
<wsdl:operation name="ValidateUser">
```

```
<soap:operation soapAction="urn:VTMS:ValidateUser"/>
```

```
<wsdl:input>
```

```
<soap:body use="literal"/>
```

```
</wsdl:input>
```

```
<wsdl:output>
```

```
<soap:body use="literal"/>
```

```
</wsdl:output>
```

```
</wsdl:operation>
```

```
<wsdl:operation name="HealthCheck">
```

```
<soap:operation soapAction="urn:VTMS:HealthCheck"/>
```

```
<wsdl:input>
```

```
<soap:body use="literal"/>
```

```
</wsdl:input>
```

```
<wsdl:output>
```

```
<soap:body use="literal"/>

</wsdl:output>

</wsdl:operation>

</wsdl:binding>

<wsdl:service name="VTMSIdentityService">

  <wsdl:port name="VTMSIdentitySoapPort"
binding="tns:VTMSIdentitySoapBinding">

    <soap:address
location="https://vtms.example.go.th/soap/VTMSIdentityService"/>

  </wsdl:port>

</wsdl:service>

</definitions>
```

หมายเหตุ: ก่อนเรียกใช้ ValidateUser SOAP Service ระบบ SSO Portal ต้องเรียก Token Acquisition Service ของ VTMS เพื่อรับ AccessToken ก่อนทุกครั้ง และต้องส่ง AccessToken มากับ ValidateUserRequest โดยใช้ร่วมกับ HTTPS เท่านั้น ห้ามใช้ Username/Password ของผู้ใช้งานหรือ WS-Security UsernameToken ในขั้นตอนนี้

ภาคผนวก E: Token Service (Client Authentication SOAP Web Service)

```
<?xml version="1.0" encoding="UTF-8"?>
<definitions name="TokenService"
  targetNamespace="http://ieat.go.th/sso/token"
  xmlns:tns="http://ieat.go.th/sso/token"
  xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
  xmlns:wSDL="http://schemas.xmlsoap.org/wsdl/"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema">

  <wSDL:types>
    <xsd:schema targetNamespace="http://ieat.go.th/sso/token"
      elementFormDefault="qualified">

      <xsd:element name="GetTokenRequest">
        <xsd:complexType>
          <xsd:sequence>
            <xsd:element name="ClientId" type="xsd:string"/>
            <xsd:element name="ClientSecret" type="xsd:string"/>
            <xsd:element name="GrantType" type="xsd:string" default="client_credentials"/>
            <xsd:element name="Scope" type="xsd:string" minOccurs="0"/>
          </xsd:sequence>
        </xsd:complexType>
      </xsd:element>

      <xsd:element name="GetTokenResponse">
        <xsd:complexType>
          <xsd:sequence>
            <xsd:element name="AccessToken" type="xsd:string" minOccurs="0"/>
            <xsd:element name="TokenType" type="xsd:string" minOccurs="0"/>
          </xsd:sequence>
        </xsd:complexType>
      </xsd:element>
    </xsd:schema>
  </wSDL:types>
</definitions>
```

```
<xsd:element name="ExpiresIn" type="xsd:int" minOccurs="0"/>
<xsd:element name="ErrorCode" type="xsd:string" minOccurs="0"/>
<xsd:element name="ErrorMessage" type="xsd:string" minOccurs="0"/>
</xsd:sequence>
</xsd:complexType>
</xsd:element>
```

```
<xsd:element name="HealthCheckRequest" type="xsd:string"/>
<xsd:element name="HealthCheckResponse" type="xsd:string"/>
```

```
</xsd:schema>
</wsdl:types>
```

```
<wsdl:message name="GetTokenInput">
  <wsdl:part name="parameters" element="tns:GetTokenRequest"/>
</wsdl:message>
```

```
<wsdl:message name="GetTokenOutput">
  <wsdl:part name="parameters" element="tns:GetTokenResponse"/>
</wsdl:message>
```

```
<wsdl:message name="HealthCheckInput">
  <wsdl:part name="parameters" element="tns:HealthCheckRequest"/>
</wsdl:message>
```

```
<wsdl:message name="HealthCheckOutput">
  <wsdl:part name="parameters" element="tns:HealthCheckResponse"/>
</wsdl:message>
```

```
<wsdl:portType name="TokenServicePortType">
```

```
<wsdl:operation name="GetToken">
  <wsdl:input message="tns:GetTokenInput"/>
  <wsdl:output message="tns:GetTokenOutput"/>
</wsdl:operation>
```

```
<wsdl:operation name="HealthCheck">
  <wsdl:input message="tns:HealthCheckInput"/>
  <wsdl:output message="tns:HealthCheckOutput"/>
</wsdl:operation>
```

```
</wsdl:portType>
```

```
<wsdl:binding name="TokenServiceSoapBinding" type="tns:TokenServicePortType">
  <soap:binding style="document" transport="http://schemas.xmlsoap.org/soap/http"/>
```

```
<wsdl:operation name="GetToken">
  <soap:operation soapAction="urn:Token:GetToken"/>
  <wsdl:input>
    <soap:body use="literal"/>
  </wsdl:input>
  <wsdl:output>
    <soap:body use="literal"/>
  </wsdl:output>
</wsdl:operation>
```

```
<wsdl:operation name="HealthCheck">
  <soap:operation soapAction="urn:Token:HealthCheck"/>
  <wsdl:input>
    <soap:body use="literal"/>
```

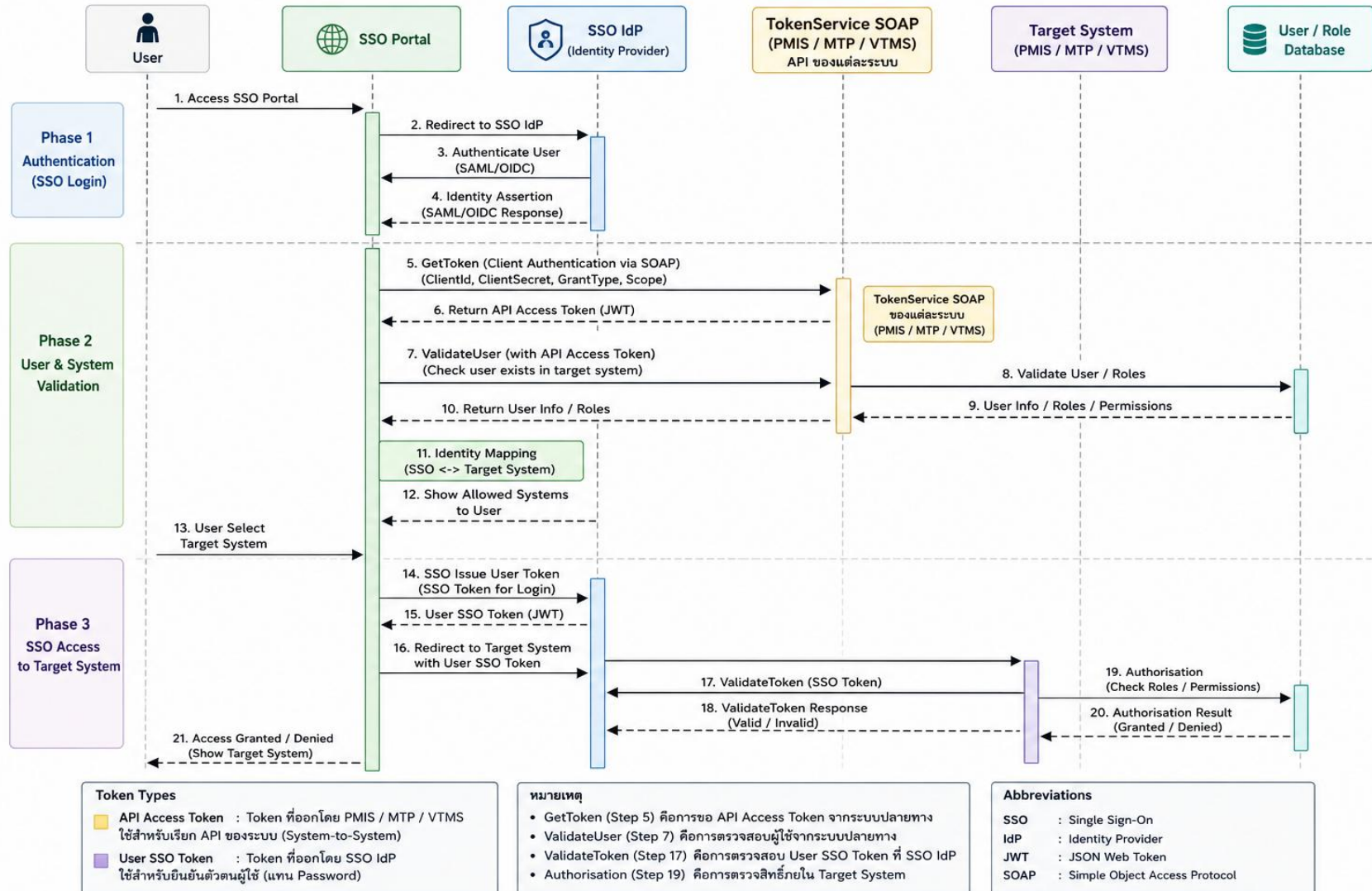
```
</wsdl:input>
<wsdl:output>
  <soap:body use="literal"/>
</wsdl:output>
</wsdl:operation>

</wsdl:binding>

<wsdl:service name="TokenService">
  <wsdl:port name="TokenServiceSoapPort" binding="tns:TokenServiceSoapBinding">
    <soap:address location="https://auth.ieatsmart.go.th/soap/TokenService"/>
  </wsdl:port>
</wsdl:service>

</definitions>
```

SSO Flow with TokenService SOAP (PMIS / MTP / VTMS)



ภาคผนวก ก. RoleMapping

System	Source Role	SSO Role	Description	Permission Scope
PMIS	PMIS_ADMIN	SSO_ADMIN	ผู้ดูแลระบบ PMIS	Full Access
PMIS	PMIS_OPERATOR	SSO_OPERATOR	เจ้าหน้าที่ปฏิบัติการ	Operate
PMIS	PMIS_VIEWER	SSO_VIEWER	ผู้ดูข้อมูล	ReadOnly
MTP	MTP_SUPERVISOR	SSO_ADMIN	Supervisor ระบบ MTP	Full Access
MTP	MTP_USER	SSO_OPERATOR	User ระบบ MTP	Operate
VTMS	VTMS_CONTROLLER	SSO_OPERATOR	ควบคุมการจราจร	Control
VTMS	VTMS_READONLY	SSO_VIEWER	ดูข้อมูล	ReadOnly

ภาคผนวก ข. CentralRole

SSORole	Description	Permission
SSO_ADMIN	ผู้ดูแลระบบ	Full Access
SSO_OPERATOR	ผู้ปฏิบัติงาน	Operate
SSO_VIEWER	ผู้ดูข้อมูล	Read Only

ภาคผนวก ค. EndpointMatrix

1. REST Endpoint Matrix

System	API	Method	Security	Description
PMIS	/api/v1/auth/token	POST	ClientId/ClientSecret	ขอ AccessToken
PMIS	/api/v1/user/validate	POST	Bearer Token	ตรวจสอบผู้ใช้งาน
MTP	/api/v1/auth/token	POST	ClientId/ClientSecret	ขอ AccessToken
MTP	/api/v1/user/validate	POST	Bearer Token	ตรวจสอบผู้ใช้งาน
VTMS	/api/v1/auth/token	POST	ClientId/ClientSecret	ขอ AccessToken
VTMS	/api/v1/user/validate	POST	Bearer Token	ตรวจสอบผู้ใช้งาน

2. SOAP Endpoint Matrix

System	SOAP Service	SOAPAction	Security	Description
PMIS	/soap/PMISIdentityService	urn:PMIS:ValidateUser	AccessToken	Validate User
MTP	/soap/MTPIdentityService	urn:MTP:ValidateUser	AccessToken	Validate User
VTMS	/soap/VTMSIdentityService	urn:VTMS:ValidateUser	AccessToken	Validate User

3. Token Service Matrix

System	Token Endpoint	Token Type	ExpiresIn
PMIS	/soap/TokenService	JWT Bearer	300
MTP	/soap/TokenService	JWT Bearer	300
VTMS	/soap/TokenService	JWT Bearer	300

4. Security Matrix

Component	Authentication	Authorisation
SSO Portal	SSO IdP	N/A
PMIS API	AccessToken	Role
MTP API	AccessToken	Role
VTMS API	AccessToken	Role
Target System	User SSO Token	Business Permission

5. SSO Portal Endpoint Matrix

Base URL: <https://dsp.ieat.go.th>

Installation Server: 192.168.10.207

System	API	Method	Security	Description
SSO	/api/v1/auth/login	POST	IdP / SAML / OIDC	รับผล Login จาก IdP และสร้าง Session
SSO	/api/v1/auth/token	POST	Bearer Token	ออก User SSO Token / Deep Link Token
SSO	/api/v1/auth/validate	POST	Token	ให้ระบบปลายทางตรวจสอบ User SSO Token
SSO	/health	GET	Internal / Allowlist	ตรวจสอบสถานะระบบ SSO

ภาคผนวก ง. CredentialMatrix

System	Credential Type	Credential Name	Usage	Mandatory
SSO	JWT Signing Key	SSO_JWT_SECRET	Sign User SSO Token / Deep Link Token	YES
SSO	TLS Certificate	SSO_TLS_CERT	HTTPS Security	YES
PMIS	Client Credential	PMIS_CLIENT_ID	GetToken / Token Acquisition	YES
PMIS	Client Secret	PMIS_CLIENT_SECRET	GetToken / Token Acquisition	YES
MTP	Client Credential	MTP_CLIENT_ID	GetToken / Token Acquisition	YES
MTP	Client Secret	MTP_CLIENT_SECRET	GetToken / Token Acquisition	YES
VTMS	Client Credential	VTMS_CLIENT_ID	GetToken / Token Acquisition	YES
VTMS	Client Secret	VTMS_CLIENT_SECRET	GetToken / Token Acquisition	YES
PMIS/MTP/VTMS	AccessToken	Runtime Token	ใช้เรียก ValidateUser	YES

